Forritunarmál
Heimaverkefni 2: Scheme

Þetta forritunarverkefni samanstendur af fjórum hlutum og heildarfjöldi stiga 50. Sérhverju falli, sem þið eigið að útfæra, fylgja einhver prófunartilvik en þið ættuð að sjálfsögðu að bæta við eigin prófunartilvikum. Þróið lausnirnar ykkar í DrRacket.

Þið megið EKKI nota innbyggð föll nema grunnföll eins *`car`*, *`cdr`*, *`cons`*, *`eq?`*, *`null?`*, *`list`*, *`if`*, *`cond`*, *`not`* og *`let`* í útfærslunni nema að það sé tiltekið sérstaklega. Þið megið þó nota æðri föll eins og *`map`* en EKKI *`apply`*.

This programming assignment comprises four parts and the total number of points is 50. Each functions, which you need to implement, is accompanied by some test cases, but you should of course add your own test cases. Develop your solutions in DrRacket.

You are NOT allowed to use built-in function, except base function like `car`, `cdr`, `cons`, `eq?`, `null?`, `list`, `if`, `cond`, `not` and `let`, in your implementation unless it is specifically allowed. However, you can use higher-order functions like `map` but NOT `apply`.

1. (20 stig) Ýmis föll / Various functions

a. (3 stig) Skrifið fallið `countdown` sem tekur heiltölu `i`>= 1 sem viðfang og skilar listanum (n n-1 ... 2 1).

Write the function `countdown` which takes an integer `i`>= 1 as an argument and returns the list (n n-1 ... 2 1).

Prófunartilvik/Test case:

```
(countdown 5)
(5 4 3 2 1)
```

b. (3 stig) Skrifið fallið `remove-every-second` sem fjarlægir annað hvert stak í viðfangslistanum. Fyrir gefinn lista (a_1 a_2 ... a_n) skilar fallið listanum (a_1 a_3... a_k), þar sem k = n ef n er oddatala, annars k = n-1.

Write the function `remove-every-second` which removes every other element of its parameter list. Given the list (a_1 a_2 ... a_n), the function returns the list (a_1 a_3... a_k), where k = n if n is odd, otherwise k = n-1.

Prófunartilvik/Test case:

```
(remove-every-second '(a b c d))
(a c)

(remove-every-second '(a b d e f))
(a d f)
```

c. (3 stig) Skrifið fallið `insert-at` sem tekur lista `lis`, heiltölu `i`>= 1 og stak `elem` sem viðföng og skilar afriti af listanum `lis` með `elem` sett inn í stöðu `i` (fyrsta staðan er `i=1`). Ef `i` > length(`lis`)+1 þá skal skila `lis` óbreyttum. Þið megið nota innbyggða fallið `length` í lausnina ykkar.

Write the function `insert-at` that, given a list `lis`, an integer `i`>= 1, and an element `elem`, returns a copy of `lis` with `elem` inserted at position `i` (the first position is `i=1`). If `i` > length(`lis`)+1, return `lis` unchanged. You are allowed to use the built-in function `length` in your solution.

Prófunartilvik/Test cases:

```
(insert-at '(a b c d) 3 'z)
(a b z c d)

(insert-at '(a b c d) 5 'z)
(a b c d z)

(insert-at '(a b c d) 6 'z)
(a b c d)
```

d. (3 stig) Skrifið fallið `sequence` sem tekur heiltölu `n` >= 1 sem viðfang og skilar listanum (1 2 … n-1 n). **Ábending**: Þið þurfið á hjálparfalli að halda.

Write the function `sequence` which takes an integer `n` >= 1 as argument and returns the list (1 2 … n-1 n). **Hint**: You need a helper function.

<u>Prófunartilvik/Test cases:</u>

```
(sequence 6)
(1 2 3 4 5 6)

(sequence 1)
'(1)
```

e. (4 stig) Skrifið fallið `combine` sem tekur þrjú viðföng: `fun` (fall), `init` (upphafsgildi) og `lis` (listi). Fallið sameinar stök listans `lis` í eitt gildi með því að nota `fun` og upphafsgildið `init`. Ef listinn `lis` er tómur þá skilar `combine init`.

f. Write the function `combine` which takes three arguments: `fun` (a function), `init` (an initial value), and `lis` (a list). The function combines the elements of the list `lis` into a single value using `fun` and the initial value `init`. If the list `lis` is empty, then `combine` returns `init`.

<u>Prófunartilvik/Test cases:</u>

```
(combine + 0 '(1 2 3 4 5))
15

(combine * 1 '(1 3 5 2 4))
120

(combine max 0 '(1 3 5 2 4))
5
```

Note that (if the list `lis` is not empty) your implementation MUST apply the function `fun` to the initial value `init` and the first element of the list `lis`, e.g.:

```
(combine + 0 '(1 2 3 4 5))
→ (combine + (+ 0 1) '(2 3 4 5))
→ (combine + (+ 1 2) '(3 4 5))
→ (combine + (+ 3 3) '(4 5))
→ (combine + (+ 6 4) '(5))
→ (combine + (+ 10 5) '())
→ 15
```

g. (4 stig) Skrifið fallið `split` sem tekur inn bool fall `fun` og lista `lis` sem viðföng. `split` skilar lista af tveimur listum (lis_1 lis_2) þar sem lis_1 inniheldur stök úr `lis` sem gera `fun` true og lis_2 inniheldur stök `elem` úr `lis` sem gera `fun` false. **Ábending**: Við þurfið á hjálparfalli að halda.

Write the function `split` which takes a boolean function `fun` and a list `lis` as arguments. `split` returns a list of two lists (lis_1 lis_2) where lis_1 contains elments from `lis` which satisfy the predicate `fun` and lis_2 contains elements from `lis` which do not satisfy the predicate `fun`. **Note**: You need a helper function.

<u>Prófunartilvik/Test cases:</u>

Note that the elements in the two output lists (lis_1 lis_2) appear in a reverse order compared to the input list `lis`.

```
(split even? '(1 2 3 4 5 6))
((6 4 2) (5 3 1))

(split (lambda(x) (> x 2)) '(1 2 3 4 5 6))
((6 5 4 3) (2 1))
```

2. (10 stig) **Æðri föll / Higher-order functions**

a. (3 stig) Skrifið fallið `make-linear` sem tekur tölur `a` og `b` sem viðföng og skilar nýju falli f(x) = `a`*x + `b`

Write the function `make-linear` which accepts the numbers `a` and `b` as arguments and returns a new function f(x) = `a`*x + `b`

Prófunartilvik/Test cases:

```
(define f (make-linear 2 1))
(f 5)
11

(define g (make-linear 3 4))
(g 5)
19
```

b. (3 stig) Skrifið fallið `add-linear` sem tekur línuleg föll `f` og `g` sem viðföng og skilar nýju falli h(x) = `f` (x) + `g` (x).

Write the function `add-linear` which accepts linear functions `f` and `g` as arguments and returns a new function h(x) = `f` (x) + `g` (x).

Prófunartilvik/Test cases:

```
((add-linear f g) 5)
30
```

c. (4 stig) Skrifið fallið `make-linear-list` sem tekur tölur `a` og `b` sem viðföng og skilar falli sem tekur lista `lis` sem viðfang og betir fallinu f(x) = `a`*x + `b` á sérhvert stak í `lis`.

Write the function `make-linear-list` which accepts numbers `a` and `b` as arguments and returns a function which accepts a list `lis` as an argument and applies the function f(x) = `a`*x + `b` to each element of `lis`.

Prófunartilvik/Test cases:

```
(define h (make-linear-list 2 1))
(h '(1 2 3))
(3 5 7)
```

3. (10 stig) **Selection sort**

   a. (3 stig) Skrifið fallið `delete-elem` sem tekur tvö viðföng: stakið `elem` og listann `lis`. Fallið skilar nýjum lista þar sem fyrsta tilvikinu af `elem` hefur verið eytt úr `lis`.

   Write the function `delete-elem` which accepts two arguments: the element `elem` and the list `lis`. The function returns a new list, in which the first occurrence of `elem` has been removed from `lis`.

   Prófunartilvik / Test cases:

```
(delete-elem 1 '(1 2 3 4))
'(2 3 4)

(delete-elem 3 '(1 2 3 4 3))
'(1 2 4 3)
```

   b. (4 stig) Skrifið fallið `select-min` sem skilar minnsta stakinu úr viðfangalistanum `lis`. **Ábending**: Notið let setningu til að finna minnsta stakið úr halanum og berið það saman við hausinn.

   Write the function `select-min` which returns the smallest element from the parameter list `lis`. **Hint**: Use a let statement to find the smallest element in the tail and compare it to the head.

   Prófunartilvik / Test cases:

```
(select-min '(4 2 1 5 7))
1
```

   c. (3 stig) Skrifið fallið `selection-sort` sem tekur inn lista `lis` sem viðfang og skilar nýjum lista, raðaðri útgáfu af `lis` (í hækkandi röð). Fallið skal nota bæði `delete-item` úr a) og `select-min` úr b).

   Write the function `selection-sort` which accepts a list `lis` as an argument and returns a new list, a sorted version of `lis` (in ascending order). The function must use both `delete-item` from a) and `select-min` from b).

```
(selection-sort '(3 5 2 1 2 5 4 6))
(1 2 2 3 4 5 5 6)
```

4. (10 stig) **Stafli / Stack**

Consider the following definition:

```
(define (make-stack stack)
   (lambda (command . args)
      (cond
           ((eq? command 'empty) (null? stack))
           (else 'error)

      )))
```

`make-stack` is a function that takes one argument, a list. The function returns an **anonymous** function (a lambda function) which itself takes at least one argument, `command`, and possibly more. If there are more arguments, they are accessible in the `args` parameter (a list).

Your task is to finish the above implementation of `make-stack`, such that, for example, the following can be executed:

```
(define stack1 (make-stack '()))
(stack1 'empty)
(stack1 'bla)
(stack1 'top)
(stack1 'pop)
(stack1 'push 2)
(stack1 'push 3)
(stack1 'top)
(stack1 'pop)
(stack1 'pop)
```

Resulting in the output:

```
#t
error
error
error
3
3
2
```

Another example:

```
(define stack2 (make-stack '(a b c)))
(stack2 'pop)
```

Resulting in the output:

```
a
```

Thus, your task is to finish the implementation of the `make-stack` function by adding the following commands:

- `top`: (3 stig) Returns error if the stack is empty, otherwise returns the top element of the stack
- `push`: (4 stig) Pushes the first element of the args list onto the top of the stack.
- `pop`: (3 stig) Returns error if the stack is empty, otherwise pops (removes) the top element of the stack and returns it.

Note that you can use the `set!` function in `push` and `pop` to change the `stack`. The syntax is `(set! variable expression)`

Leiðbeiningar varðandi skil / *Instructions for submission*:

Skrifið og prófið föllin ykkar í DrRacket í skrá `project2.rkt`. Þegar þið sendið lausnina ykkar inn í Gradescope þá þurfið þið að búa til nýja skrá, `project2.scm` (og eingöngu skila henni), sem er eins og `project2.rkt` en með eftirtöldum tveimur línum efst:

Develop and test your functions in DrRacket in a file `project2.rkt`. When you submit your solution into Gradescope, you need to create a new file, `project2.scm` (and only sbmit that file), which is identical to `project2.rkt`, but with the following two lines at the top of the file:

```
(declare (unit project2))
(import (chicken sort))
```

Ástæðan er sú að autograder í Gradescope notar Chicken Scheme (https://call-cc.org/) en ekki DrRacket. Athugið jafnframt að `project2.scm` skráin má ekki innihalda neinar Racket metadata línur.

The reason is the Gradescope uses Chicken Scheme (https://call-cc.org/), but not DrRacket. Note also that the `project2.scm` file can not contain any Racket metadata lines.